Persistence

Juan-Luis Gorricho

Creating a data base

- Using a data base assistant:
  - Create a new data base (schema): sample
  - Create new tables for the data base.
  - Define different fields on each table. The first one always:
    - id of type int, primary key, not null and auto-increment
  - Give user: ecomm/ecomm privileges with access granted to the data base and full permissions

Defining the DDBB – Applicaton Server connection

- At netBeans Services / Databases, create a new connection. Choose:
  - Name: MySQL
  - Host: localhost
  - Port: 3306
  - Database: database name (sample)
  - User / password: ecomm / ecomm

Entity classes

- Create a new web application project
- Create an entity package at source packages
- Create new entity classes from database at the new entity package:
  - Select from data source the previously created new database connection
  - Select all tables on database to create the entity classes

Persistence unit

- A persistence unit defines a set of all entity classes that are managed by **EntityManager** instances in an application. This set of entity classes represents the data contained within a single data store.
- Persistence units are defined by the **persistence.xml** configuration file. The JAR file or directory whose META-INF directory contains persistence.xml is called the root of the persistence unit.

ContextListener set up

- Create a web application listener:

```
public class ContextListener implements ServletContextListener {
    @Resource
    public UserTransaction utx;

    @PersistenceContext
    public EntityManager em;

    public void contextInitialized(ServletContextEvent sce) {
        ServletContext context = sce.getServletContext();
        context.setAttribute("utx", utx);
        context.setAttribute("em", em);
        …
```

CRUD operations

- Create:

```
Customer record   = new Customer();
record.setName("Peter");
record.setSurname("Johnson");
utx.begin();
em.persist(record);
utx.commit();
```

- Retrieve:

```
Table_name   record   = em.find(Table_name.class, id);
```

CRUD operations

- Update:

```
record.setTitle("anotherTitle");
…
utx.begin();
em.merge(record);
utx.commit();
```

- Delete:

```
utx.begin();
record = em.merge(record);
em.remove(record);
utx.commit();
```

Creating queries

- SELECT statement example:

```
Query query = em.createQuery("select o from Customer as o where
    o.login=:login and o.password=:password");

query.setParameter("login", “john”);
query.setParameter("password", “smith”);

List<Customer> customers = query.getResultList();

Customer customer = ((Customer)query.getSingleResult());
```

SELECT statements

- A select query has six clauses: SELECT, FROM, WHERE, GROUP BY, HAVING, and ORDER BY. The SELECT and FROM clauses are required, but the WHERE, GROUP BY, HAVING, and ORDER BY clauses are optional. Here is the high-level BNF syntax of a query language query:

  QL_statement ::= select_clause from_clause [where_clause] [group by_clause] [having_clause] [order by_clause]

- The SELECT clause defines the types of the objects or values returned by the query.
- The FROM clause defines the scope of the query by declaring one or more identification variables, which can be referenced in the SELECT and WHERE clauses.
- The WHERE clause is a conditional expression that restricts the objects or values retrieved by the query. Although it is optional, most queries have a WHERE clause.
- The GROUP BY clause groups query results according to a set of properties.
- The HAVING clause is used with the GROUP BY clause to further restrict the query results according to a conditional expression.
- The ORDER BY clause sorts the objects or values returned by the query into a specified order.

UPDATE and DELETE statements

- Update and delete statements provide bulk operations over sets of entities. They have the following syntax:

update_statement :: = update_clause [where_clause]
delete_statement :: = delete_clause [where_clause]

- The update and delete clauses determine the type of the entities to be updated or deleted. The WHERE clause may be used to restrict the scope of the update or delete operation.

update and delete statements examples

```
utx.begin();
Query q = em.createQuery("delete from Customer as o where o.login=:login");
query.setParameter("login", “john”);
int tmp = q.executeUpdate();
em.flush();
utx.commit();

utx.begin();
Query q = em.createQuery(“update Customer as o set o.phone=:phone where
     o.login=:login");
query.setParameter("phone", “934016000”);
query.setParameter("login", “john”);
int tmp = q.executeUpdate();
em.flush();
utx.commit();
```

other SELECT statements

- getSingleResult() and/or corresponding return type value when appropriate:

Query q = em.createQuery("select distinct o.column_name from Customer as o”);

Query q = em.createQuery("select count(o) from Customer as o”);

Query q = em.createQuery("select max(o.column_name) from Customer as o");

Exercises

- Import the demoddbb.sql data base
- Open the demoBBDD_exercise
- Uncomment and test the necessary code at the servlet to:
  - Create a new Customer
  - Update a Customer
  - Retrieve and delete a Customer
  - Display all customers with a given name
  - Etc.